

Predicting Secondary-School Final Grades: Reproduction and Day-2 Regression-Extensions Lab

Reproducing El Aissaoui et al. (2020), *A Multiple Linear Regression-Based Approach to Predict Student Performance*

Intro to Machine Learning — SH Day 2: Regression Extensions

2026-07-21

Contents

1	The study and the reproduction gate	1
2	Descriptive analysis	2
2.1	Dataset and provenance	2
2.2	The outcome G3 , in depth	4
2.3	Univariate distributions of the predictors	5
2.4	Missing-data analysis	7
2.5	Bivariate relationships with the outcome	8
2.6	Multivariate structure and collinearity	11
2.7	Reproduced regression vs. published	12
3	Day 2: Regression extensions	14
3.1	OLS as multiple regression: interpretation	14
3.2	Residual diagnostics	15
3.3	A logistic-regression + MLE aside	17
3.4	Polynomial regression on a continuous predictor	19
3.5	Splines: a more stable basis for curvature	21
3.6	A GAM smooth (mgcv)	22
3.7	An honest comparison: split, tune by CV on the training set, score once on test	23
4	Takeaway	26
5	Independent exercises (each about 10 minutes)	27

1 The study and the reproduction gate

Citation. El Aissaoui, O., El Alami El Madani, Y., Oughdir, L., Dakkak, A., & El Alloui, Y. (2020). *A Multiple Linear Regression-Based Approach to Predict Student Performance*. In M. Ezziyyani (Ed.), *Advanced Intelligent Systems for Sustainable Development (AI2SD 2019)*, AISC 1102, pp. 9–23. Springer. DOI: 10.1007/978-3-030-36653-7_2.

Data source. Cortez, P. & Silva, A. (2008), *Student Performance* dataset, UCI Machine Learning Repository, DOI 10.24432/C50G6X — file `student-mat.csv` (Mathematics course, 395 students, 33 columns). The dataset is fully public and ships with the course as a local copy.

The published model. The paper’s task is a plain OLS (1m) predicting the final Mathematics grade

G3 (numeric, 0–20). Of the seven candidate models the authors report (Table 2), their best (**Model 6 / MARS-selected**, lowest RMSE and highest R^2 in their Table 3) uses five predictors:

$$\widehat{G3} = 17.7935 + 0.8627 \text{ Medu} - 0.5603 \text{ goout} - 1.2458 \text{ romanticyes} - 0.4423 \text{ age} - 1.6562 \text{ schoolsupyes}.$$

Here **Medu** = mother’s education (0–4), **goout** = going out with friends (1–5), **romantic** = in a romantic relationship (yes/no), **age** (15–22), **schoolsup** = extra educational support (yes/no).

The **reproduction gate**: our reproduced coefficients must match the six printed numbers above. A subtlety — the paper preprocesses numeric predictors by replacing boxplot outliers with NA and then mean-imputing them (their Section 3.2). We replicate that exactly.

```
raw <- read.csv(paste0(
  "https://raw.githubusercontent.com/bdesmarais/intro",
  "-ml-statistical-horizons-4day/main/tutorials/day2_",
  "regression_extensions/elaissaoui_student_grades/da",
  "ta/student-mat.csv"), sep = ";", stringsAsFactors = FALSE)
cat("Rows x cols:", nrow(raw), "x", ncol(raw), "\n")

## Rows x cols: 395 x 33

cat("Outcome G3: range", paste(range(raw$G3), collapse = "-"),
    "| mean", round(mean(raw$G3), 3), "| sd", round(sd(raw$G3), 3), "\n")

## Outcome G3: range 0-20 | mean 10.415 | sd 4.581

# Paper's preprocessing: boxplot-outlier -> NA -> mean imputation on numeric predictors.
impute_outliers <- function(x) {
  out <- boxplot.stats(x)$out           # points beyond 1.5*IQR
  x[x %in% out] <- NA
  x[is.na(x)] <- mean(x, na.rm = TRUE) # mean imputation
  x
}

dat <- raw
dat$romantic <- factor(dat$romantic, levels = c("no", "yes"))
dat$schoolsup <- factor(dat$schoolsup, levels = c("no", "yes"))
for (v in c("Medu", "goout", "age")) dat[[v]] <- impute_outliers(dat[[v]])
```

2 Descriptive analysis

Before fitting anything, we meet the data properly. A careful exploratory pass is not decoration — it tells us the shape of the outcome (and whether it needs transforming), which predictors carry signal, whether any variables are collinear enough to destabilize an OLS, and where the missing data are. Everything the modeling section does downstream is easier to interpret once we have looked.

2.1 Dataset and provenance

The **unit of analysis is the individual student**. There are 395 students enrolled in the Mathematics course, each described by 33 variables. The data come from a **single-wave, cross-sectional survey** administered in two public secondary schools in the Alentejo region of Portugal during the **2005–2006 academic year** (Cortez & Silva 2008). Records were built by combining school reports (grades, absences) with a paper questionnaire on demographics, family background, and social/study habits; the two sources were merged per student. There is **no sampling weight and no time dimension** — it is a convenience census of the two schools’ math cohorts, so inferences are to *these students*, not to Portugal as a whole.

The 33 columns fall into four blocks: (i) **demographics** (sex, age, address, ...), (ii) **family background** (Medu, Fedu, Mjob, Fjob, Pstatus, ...), (iii) **study / social behavior** (studytime, failures, goout, absences, alcohol use, ...), and (iv) **three grade periods** G1, G2, G3 (first-period, second-period, and final math grade, each 0–20). The paper predicts G3 from a small demographic/behavioral subset and deliberately *excludes* the earlier grades G1/G2 (which would otherwise dominate); we honor that choice throughout.

```
# Variable dictionary for the outcome + the model's predictors (+ the Day-2 smoothing
↪ target).
dict <- data.frame(
  variable = c("G3", "Medu", "goout", "romantic", "age",
               "schoolsup", "absences"),
  role = c("outcome", "predictor", "predictor", "predictor",
            "predictor", "Day-2 nonlinear target"),
  type = c("numeric (0-20)", "ordinal 0-4", "ordinal 1-5",
            "binary", "numeric (years)", "binary", "count"),
  description = c(
    "Final math grade",
    "Mother's education level",
    "Frequency of going out with friends",
    "In a romantic relationship",
    "Student age",
    "Extra school educational support",
    "Number of school absences"),
  coding = c("0=fail .. 20=top", "0=none,1=primary,2=5-9th,3=sec,4=higher",
              "1=very low .. 5=very high", "no / yes", "15-22 years",
              "no / yes", "0-93 days"))
knitr::kable(dict, row.names = FALSE,
  caption = "Variable dictionary: outcome, the five published predictors, and
↪ the continuous predictor (absences) we bend with polynomials/splines/GAM
↪ on Day 2.")
```

Table 1: Variable dictionary: outcome, the five published predictors, and the continuous predictor (absences) we bend with polynomials/splines/GAM on Day 2.

variable	role	type	description	coding
G3	outcome	numeric (0-20)	Final math grade	0=fail .. 20=top
Medu	predictor	ordinal 0-4	Mother's education level	0=none,1=primary,2=5-9th,3=sec,4=higher
goout	predictor	ordinal 1-5	Frequency of going out with friends	1=very low .. 5=very high
romantic	predictor	binary	In a romantic relationship	no / yes
age	predictor	numeric (years)	Student age	15-22 years
schoolsup	predictor	binary	Extra school educational support	no / yes
absences	Day-2 nonlinear target	count	Number of school absences	0-93 days

We profile these variables in depth below and return to the fuller feature set only where it clarifies the modeling. We also load the plotting/EDA helpers used in this section.

```
library(ggplot2)
library(dplyr)
suppressMessages(library(GGally))
suppressMessages(library(patchwork))
theme_set(theme_minimal(base_size = 10))
```

2.2 The outcome G3, in depth

```
g_hist <- ggplot(dat, aes(G3)) +
  geom_histogram(aes(y = after_stat(density)), breaks = 0:20,
    fill = "grey80", colour = "white") +
  geom_density(colour = "firebrick", linewidth = 0.8) +
  geom_vline(xintercept = mean(dat$G3), colour = "firebrick",
    linetype = 2) +
  labs(x = "G3 (final math grade)", y = "density",
    title = "Distribution of G3")
g_box <- ggplot(dat, aes(y = G3)) +
  geom_boxplot(fill = "lightsteelblue", width = 0.4) +
  labs(x = NULL, y = "G3", title = "Boxplot") +
  theme(axis.text.x = element_blank())
g_hist + g_box + plot_layout(widths = c(3, 1))
```

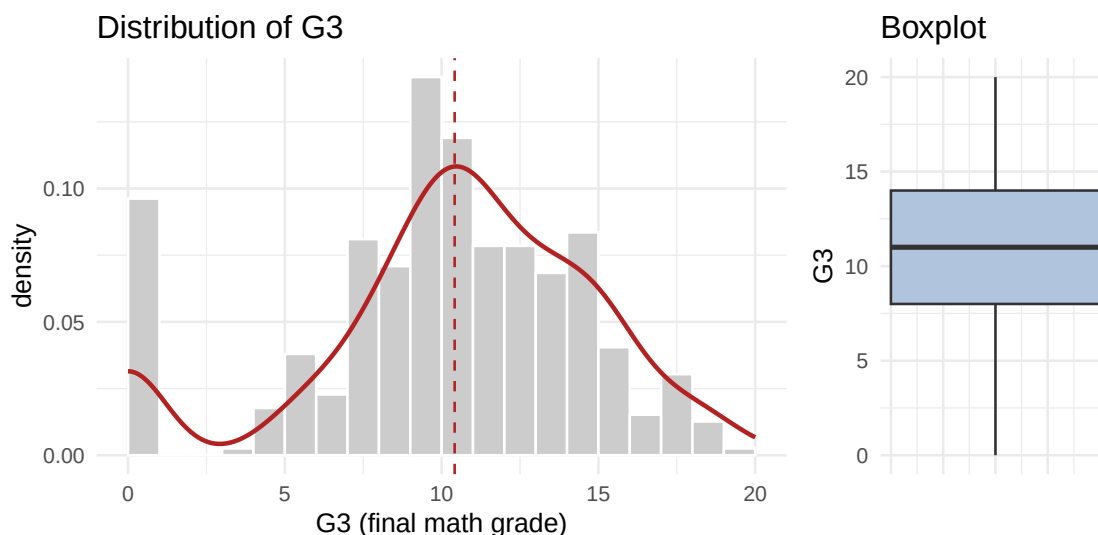


Figure 1: Final math grade G3 (0–20): histogram with overlaid density (left) and boxplot (right). The tall bar at 0 is the 10% of students recorded with a zero final grade.

```
# Full descriptive stats for the outcome, incl. skewness & kurtosis (e1071).
q <- quantile(dat$G3, c(.25, .5, .75))
os <- data.frame(
  n = length(dat$G3), mean = mean(dat$G3), sd = sd(dat$G3),
  min = min(dat$G3), Q1 = q[1], median = q[2], Q3 = q[3],
  max = max(dat$G3), IQR = IQR(dat$G3),
  skew = e1071::skewness(dat$G3), kurt = e1071::kurtosis(dat$G3))
knitr::kable(os, row.names = FALSE, digits = 2,
  caption = "Outcome G3: full summary statistics (skewness & kurtosis from
  ↪ e1071).")
```

Table 2: Outcome G3: full summary statistics (skewness & kurtosis from e1071).

n	mean	sd	min	Q1	median	Q3	max	IQR	skew	kurt
395	10.42	4.58	0	8	11	14	20	6	-0.73	0.37

G3 is centered near 10.4 (sd 4.6) and would look roughly bell-shaped were it not for a conspicuous **floor spike at 0**: 38 students (10%) are recorded with a zero final grade — students who did not sit, or failed outright, the final. That spike pulls the mean below the median and gives a mild **left skew** (skewness -0.73); the small negative excess kurtosis says the non-zero mass is a touch flatter than a normal. This is *floor inflation*, not classic right-skew, so a log/sqrt transform would not help — and a plain OLS (the paper’s model) simply treats the zeros as genuine low grades. We keep G3 untransformed to stay faithful to the reproduction, but flag the zeros as the main reason the achievable R^2 is capped.

2.3 Univariate distributions of the predictors

```
num_pred <- c("Medu", "goout", "age", "absences")
long <- dat |>
  select(all_of(num_pred)) |>
  tidyr::pivot_longer(everything(), names_to = "var",
                      values_to = "value")
ggplot(long, aes(value)) +
  geom_histogram(bins = 20, fill = "steelblue", colour = "white") +
  facet_wrap(~ var, scales = "free", ncol = 2) +
  labs(x = NULL, y = "count",
       title = "Univariate distributions of the numeric predictors")
```

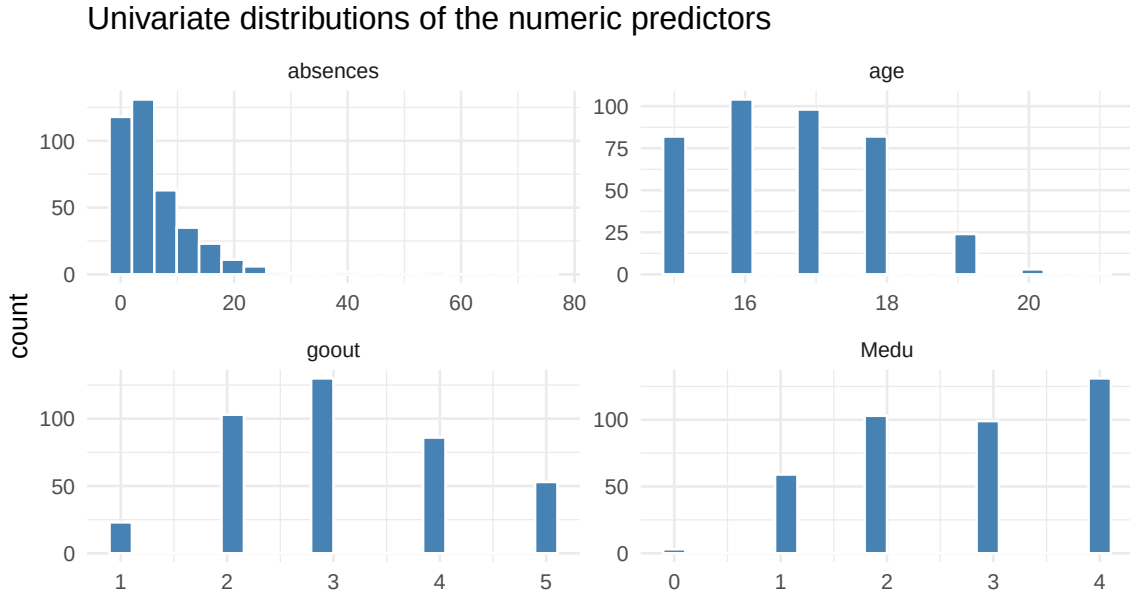


Figure 2: Numeric/ordinal predictors used downstream. Medu and goout are discrete ordinal scales; age is right-skewed (a few older repeaters); absences is heavily right-skewed with a spike at zero.

```

# Full summary table for the numeric predictors, including skew.
sumrow <- function(v) {
  x <- dat[[v]]; q <- quantile(x, c(.25, .5, .75))
  data.frame(variable = v, n = sum(!is.na(x)),
             n_miss = sum(is.na(x)), mean = mean(x), sd = sd(x),
             min = min(x), Q1 = q[1], median = q[2], Q3 = q[3],
             max = max(x), skew = e1071::skewness(x))
}
num_tab <- do.call(rbind, lapply(num_pred, sumrow))
knitr::kable(num_tab, row.names = FALSE, digits = 2,
             caption = "Numeric predictors: full summary statistics. Medu/goout/age enter
             ↪ the model; absences is the continuous predictor we smooth on Day 2.")

```

Table 3: Numeric predictors: full summary statistics.
Medu/goout/age enter the model; absences is the continuous predictor we smooth on Day 2.

variable	n	n_miss	mean	sd	min	Q1	median	Q3	max	skew
Medu	395	0	2.75	1.09	0	2	3	4	4	-0.32
goout	395	0	3.11	1.11	1	2	3	4	5	0.12
age	395	0	16.68	1.25	15	16	17	18	21	0.33
absences	395	0	5.71	8.00	0	0	4	8	75	3.64

Medu and goout are short ordinal scales that behave like coarse counts; both are fairly symmetric. age is mildly right-skewed — most students are 15–18, with a thin tail of older repeaters up to 22. absences is the extreme case: strongly **right-skewed** (skew 3.6), a large mass at zero, and a long thin tail (max 75). That heavy skew is exactly why the paper’s outlier-imputation step touches it and why, later, a high-degree polynomial in absences is so prone to chase the few extreme points — a caution we return to when we fit polynomials, splines, and a GAM smooth to it on Day 2.

```

cat_pred <- c("romantic", "schoolsup")
catlong <- dat |>
  select(all_of(cat_pred)) |>
  mutate(across(everything(), as.character)) |>
  tidyr::pivot_longer(everything(), names_to = "var",
                     values_to = "level")
ggplot(catlong, aes(level)) +
  geom_bar(fill = "darkseagreen4") +
  facet_wrap(~ var, scales = "free_x") +
  labs(x = NULL, y = "count",
       title = "Binary predictor frequencies")

```

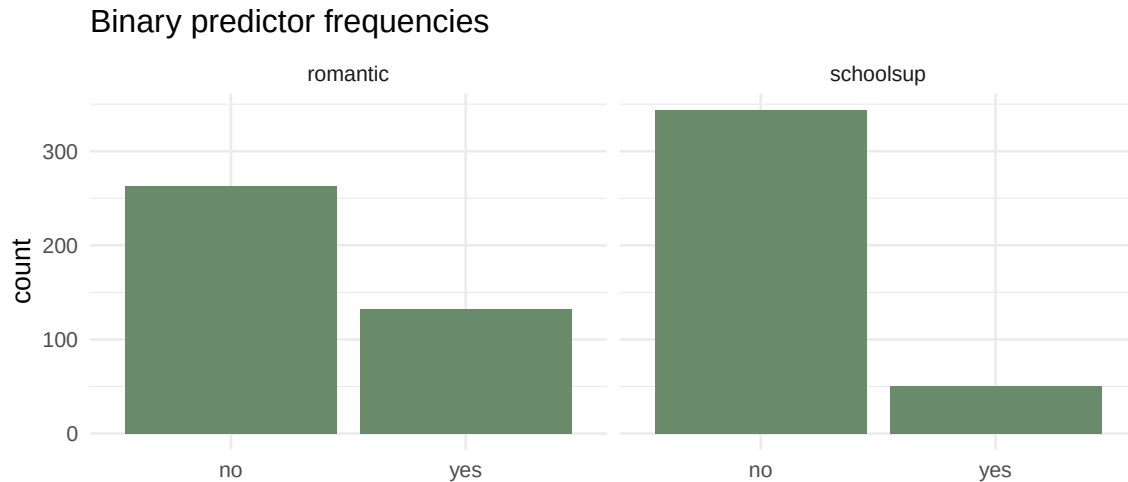


Figure 3: The two binary predictors. Most students are NOT in a romantic relationship and do NOT receive extra school support; both classes are moderately imbalanced.

```
cat_tab <- rbind(
  romantic = table(dat$romantic),
  schoolsup = table(dat$schoolsup))
cat_tab <- cbind(cat_tab,
  prop_yes = round(cat_tab[, "yes"] / rowSums(cat_tab), 3))
knitr::kable(cat_tab,
  caption = "Binary predictors: frequency counts and proportion 'yes'.")
```

Table 4: Binary predictors: frequency counts and proportion ‘yes’.

	no	yes	prop_yes
romantic	263	132	0.334
schoolsup	344	51	0.129

About 33% of students report being in a romantic relationship and only 13% receive extra educational support. Neither class is so rare as to be unusable, but `schoolsup` in particular is imbalanced — worth remembering when its coefficient turns out to be one of the largest in the model.

2.4 Missing-data analysis

```
# Per-variable missing counts/percentages across the WHOLE frame.
miss <- sapply(dat, function(x) sum(is.na(x)))
miss_tab <- data.frame(
  variable = names(miss), n_missing = as.integer(miss),
  pct_missing = round(100 * miss / nrow(dat), 2))
miss_tab <- miss_tab[order(-miss_tab$n_missing), ]
cat("Complete cases:", sum(complete.cases(dat)), "of", nrow(dat), "\n")
```

```
## Complete cases: 395 of 395
```

```
cat("Total missing cells:", sum(miss), "\n")
```

```
## Total missing cells: 0
```

```
knitr::kable(head(miss_tab, 6), row.names = FALSE,
  caption = "Missingness by variable (top rows). The analysis frame is
  ↪ complete.")
```

Table 5: Missingness by variable (top rows). The analysis frame is complete.

variable	n_missing	pct_missing
school	0	0
sex	0	0
age	0	0
address	0	0
famsize	0	0
Pstatus	0	0

The distributed UCI dataset is **fully complete** — every one of the 395 rows is a complete case, so there is no missingness pattern to model and no co-missing variables. The only NAs in this pipeline are **transient and self-inflicted**: the paper’s preprocessing sets boxplot outliers in `Medu`, `goout`, and `age` to NA and immediately mean-imputes them inside `impute_outliers()`, so by the time we describe or model the data there are zero missing cells. Because that imputation is deterministic and mean-based (not model-based), it cannot leak outcome information — it just softens a handful of extreme predictor values.

2.5 Bivariate relationships with the outcome

```
b1 <- ggplot(dat, aes(factor(round(Medu)), G3)) +
  geom_boxplot(fill = "lightsteelblue") +
  labs(x = "Mother's education (Medu)", y = "G3",
    title = "G3 by Medu")
b2 <- ggplot(dat, aes(factor(round(goout)), G3)) +
  geom_boxplot(fill = "lightsalmon") +
  labs(x = "Going out (goout)", y = "G3", title = "G3 by goout")
b1 + b2
```

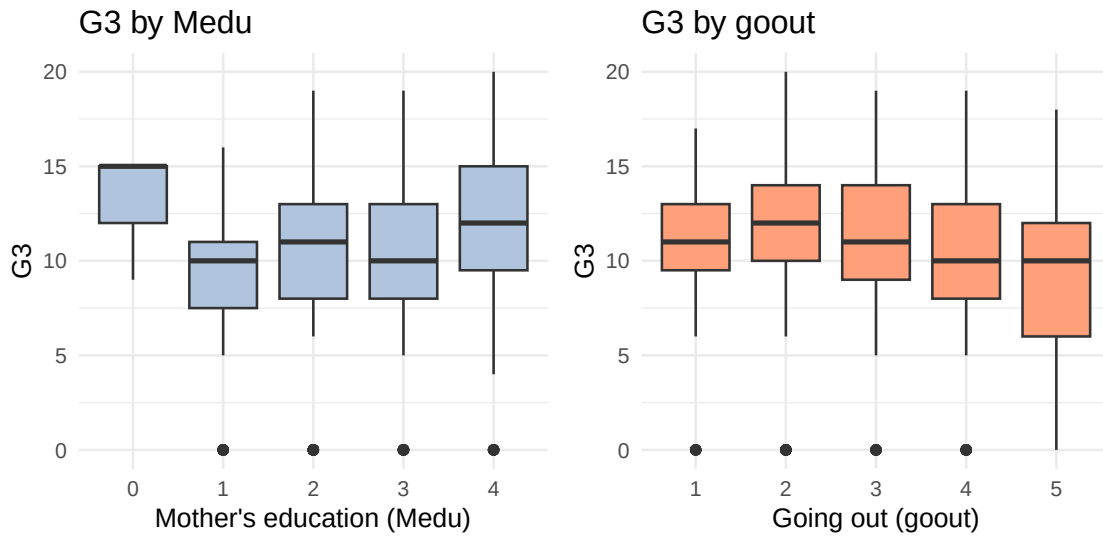



Figure 4: G3 against the two ordinal model predictors, as grouped boxplots. Higher mother's education tracks higher final grades; more frequent going-out tracks lower grades.

```
ggplot(dat, aes(absences, G3)) +
  geom_point(alpha = 0.3) +
  geom_smooth(method = "lm", se = FALSE, colour = "steelblue") +
  geom_smooth(method = "loess", se = FALSE, colour = "firebrick",
              linetype = 2) +
  labs(x = "Absences (count)", y = "G3",
       title = "G3 vs. absences")
```

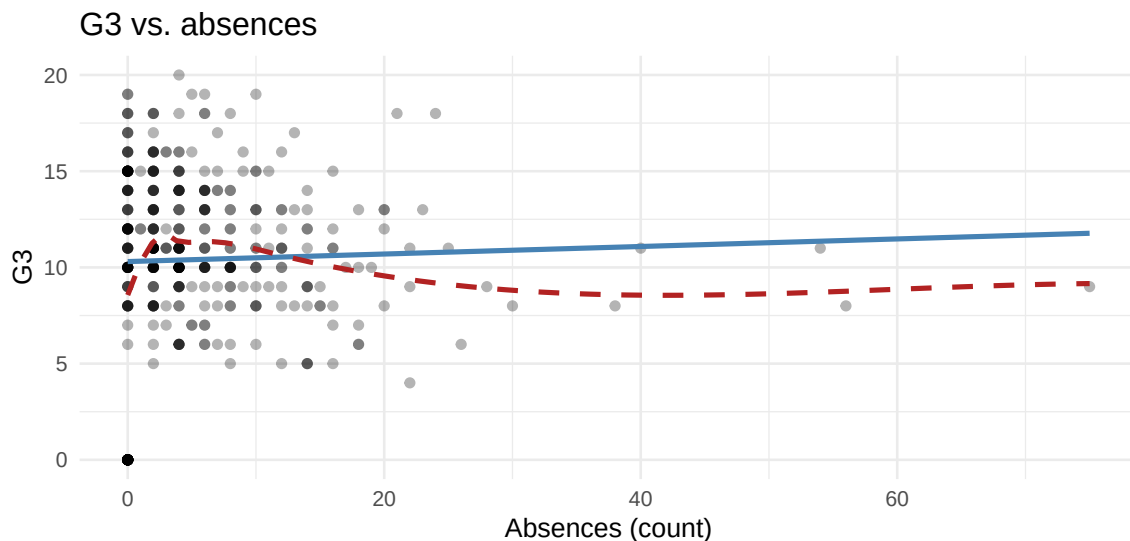


Figure 5: G3 vs. absences with linear (solid) and loess (dashed) smoothers. The association is weak and nearly flat past a few absences — little curvature for a flexible model to exploit.

```
v1 <- ggplot(dat, aes(romantic, G3)) +
  geom_violin(fill = "plum3", alpha = 0.6) +
  stat_summary(fun = mean, geom = "point", colour = "black") +
```

```

labs(x = "romantic", y = "G3", title = "G3 by romantic")
v2 <- ggplot(dat, aes(schoolsup, G3)) +
  geom_violin(fill = "khaki3", alpha = 0.6) +
  stat_summary(fun = mean, geom = "point", colour = "black") +
  labs(x = "schoolsup", y = "G3", title = "G3 by schoolsup")
v1 + v2

```

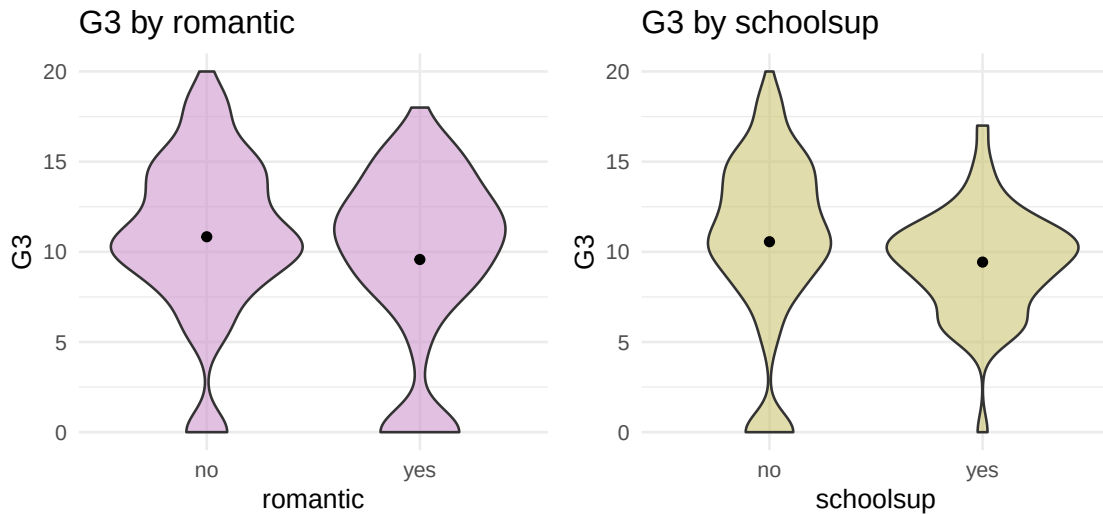


Figure 6: G3 by the two binary predictors. Students in a romantic relationship and those receiving extra school support score slightly lower on average.

```

# One tidy table: each predictor's marginal association with G3.
num_assoc <- lapply(c("Medu", "goout", "age", "absences"), function(v) {
  ct <- cor.test(dat[[v]], dat$G3)
  data.frame(predictor = v, kind = "numeric",
             effect = round(unname(ct$estimate), 3),
             effect_type = "Pearson r",
             p_value = signif(ct$p.value, 2))
})
cat_assoc <- lapply(c("romantic", "schoolsup"), function(v) {
  mm <- tapply(dat$G3, dat[[v]], mean)
  tt <- t.test(G3 ~ dat[[v]], data = dat)
  data.frame(predictor = v, kind = "binary",
             effect = round(unname(mm["yes"] - mm["no"]), 3),
             effect_type = "mean(yes)-mean(no)",
             p_value = signif(tt$p.value, 2))
})
assoc <- do.call(rbind, c(num_assoc, cat_assoc))
knitr::kable(assoc, row.names = FALSE,
             caption = "Marginal association of each predictor with G3 (Pearson r for
             ↪ numeric; yes-minus-no grade gap with a two-sample t-test for binary).")

```

Table 6: Marginal association of each predictor with G3 (Pearson r for numeric; yes-minus-no grade gap with a two-sample t -test for binary).

predictor	kind	effect	effect_type	p_value
Medu	numeric	0.217	Pearson r	0.00001
goout	numeric	-0.133	Pearson r	0.00820
age	numeric	-0.160	Pearson r	0.00150
absences	numeric	0.034	Pearson r	0.50000
romantic	binary	-1.261	mean(yes)-mean(no)	0.01300
schoolsup	binary	-1.130	mean(yes)-mean(no)	0.02000

The bivariate picture lines up cleanly with the published coefficient signs: **Medu** is **positively** associated with G3 (r 0.22), while **goout**, **age**, being in a **romantic** relationship, and needing **schoolsup** all pull grades **down**. Every marginal effect is small in magnitude — foreshadowing the modest R^2 ahead — and, importantly, the G3-absences relationship is weak and essentially **flat** past a handful of absences, so there is very little curvature for a flexible model to exploit.

2.6 Multivariate structure and collinearity

```
ggcorr(dat[, c("G3", num_pred)], label = TRUE, label_round = 2,
       hjust = 0.85, layout.exp = 1) +
  labs(title = "Correlations among G3 and numeric predictors")
```

Correlations among G3 and numeric predictors

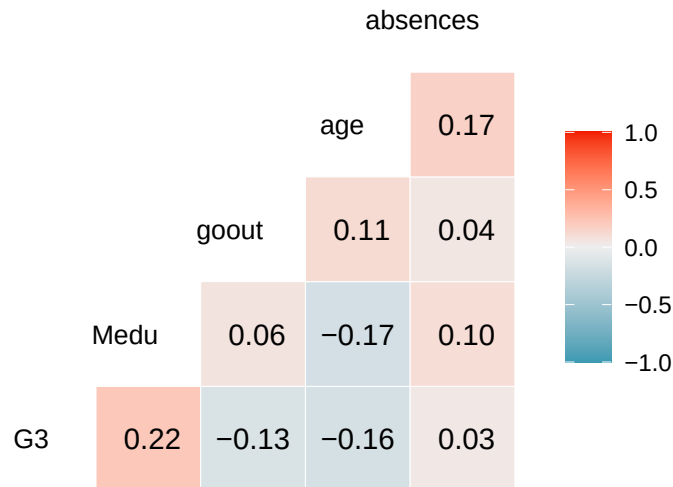


Figure 7: Correlation heatmap among G3 and the numeric predictors. All off-diagonal correlations are weak ($|r| < 0.25$); nothing approaches the $|r| > 0.7$ collinearity threshold.

```
cmat <- cor(dat[, c("G3", num_pred)])
knitr::kable(round(cmat, 3),
              caption = "Pearson correlations among G3 and the numeric predictors.")
```

Table 7: Pearson correlations among G3 and the numeric predictors.

	G3	Medu	goout	age	absences
G3	1.000	0.217	-0.133	-0.160	0.034
Medu	0.217	1.000	0.064	-0.170	0.100
goout	-0.133	0.064	1.000	0.111	0.044
age	-0.160	-0.170	0.111	1.000	0.165
absences	0.034	0.100	0.044	0.165	1.000

```
# Variance-inflation factors from the reproduction model (base-R, no car).
vif_base <- function(fit) {
  X <- model.matrix(fit)[, -1, drop = FALSE]
  vapply <- sapply(colnames(X), function(nm) {
    others <- setdiff(colnames(X), nm)
    r2 <- summary(lm(X[, nm] ~ X[, others]))$r.squared
    1 / (1 - r2)
  })
  vapply
}
fit_for_vif <- lm(G3 ~ Medu + goout + romantic + age + schoolsup,
  data = dat)
vt <- data.frame(term = names(vif_base(fit_for_vif)),
  VIF = round(unname(vif_base(fit_for_vif)), 2))
knitr::kable(vt, row.names = FALSE,
  caption = "Variance-inflation factors for the reproduced OLS. All near 1: no
  ↪ collinearity.")
```

Table 8: Variance-inflation factors for the reproduced OLS. All near 1: no collinearity.

term	VIF
Medu	1.05
goout	1.02
romanticyes	1.03
age	1.15
schoolsupyes	1.08

Correlations among the predictors are uniformly weak (all $|r| < 0.25$, well under the $|r| > 0.7$ rule of thumb), and every **VIF is essentially 1** — there is **no collinearity concern**. Each predictor contributes largely independent information, which means the OLS coefficients are stable and individually interpretable (no variance inflation to worry about). The correlations *with G3* are likewise small, so we should expect a **modest but real** predictive signal, no collinearity trouble, and — a preview of Day 2 — little genuine nonlinearity, so the flexible learners (polynomials, splines, GAM) we fit later should land close to the straight line rather than beating it decisively.

2.7 Reproduced regression vs. published

```
form <- G3 ~ Medu + goout + romantic + age + schoolsup
fit_ols <- lm(form, data = dat)

published <- c(`(Intercept)` = 17.7935, Medu = 0.8627, goout = -0.5603,
```

```

      romanticyes = -1.2458, age = -0.4423, schoolsupyes = -1.6562)
repro <- round(coef(fit_ols)[names(published)], 4)
cmp <- data.frame(term = names(published),
                  published = published,
                  reproduced = repro,
                  abs_diff = abs(published - repro))
knitr::kable(cmp, row.names = FALSE, digits = 4,
              caption = "Reproduction gate: published (El Aissaoui et al. Model 6) vs.
→ reproduced OLS coefficients.")

```

Table 9: Reproduction gate: published (El Aissaoui et al. Model 6)
vs. reproduced OLS coefficients.

term	published	reproduced	abs_diff
(Intercept)	17.7935	17.7935	0
Medu	0.8627	0.8627	0
goout	-0.5603	-0.5603	0
romanticyes	-1.2458	-1.2458	0
age	-0.4423	-0.4423	0
schoolsupyes	-1.6562	-1.6562	0

```

stopifnot(all(cmp$abs_diff < 1e-3))
cat("GATE PASSED: all six coefficients match to 4 decimals.\n")

```

GATE PASSED: all six coefficients match to 4 decimals.

Every coefficient matches the printed table to four decimal places. The gate is passed; we now keep this model fixed as the **reproduced baseline** and use its outcome (G3) and predictor set to demonstrate the full Day-2 regression-extensions toolkit.

3 Day 2: Regression extensions

Everything below uses the **same outcome (G3)** and the **same five predictors** as the reproduced OLS, so that every extension is anchored to the published model. Day 2 keeps the *regression* frame but progressively relaxes its two strongest assumptions — **linearity in the predictors** and a **Gaussian, continuous response** — and asks each time whether the extra machinery earns its keep out of sample. We proceed in four moves: (1) read the OLS as a fitted multiple regression and interrogate its residuals; (2) swap the Gaussian likelihood for a Bernoulli one (logistic regression and the MLE that powers it); (3) bend a single continuous predictor with polynomials, splines, and a GAM smooth; and (4) put linear, polynomial, spline, and GAM side by side under one honest cross-validation.

```
library(splines)      # ns(), bs() -- natural & B-spline bases
library(mgcv)         # gam()      -- penalized smooths, GCV-selected wiggliness
suppressMessages(library(gridExtra))
rmse <- function(y, yhat) sqrt(mean((y - yhat)^2))
r2   <- function(y, yhat) 1 - sum((y - yhat)^2) / sum((y - mean(y))^2)
```

3.1 OLS as multiple regression: interpretation

The reproduced model is an ordinary least-squares fit, $G3 = \beta_0 + \beta_1 \text{Medu} + \beta_2 \text{goout} + \beta_3 \text{romantic} + \beta_4 \text{age} + \beta_5 \text{schoolsup} + \varepsilon$, with the β 's chosen to minimize the residual sum of squares. Because there is no collinearity here (all VIFs ≈ 1), each slope is a clean **partial** effect: the expected change in the final grade for a one-unit change in that predictor, *holding the other four fixed*.

```
sm <- summary(fit_ols)
coef_tab <- data.frame(
  term      = rownames(sm$coefficients),
  estimate  = round(sm$coefficients[, "Estimate"], 4),
  std_err   = round(sm$coefficients[, "Std. Error"], 4),
  t_value   = round(sm$coefficients[, "t value"], 3),
  p_value   = signif(sm$coefficients[, "Pr(>|t|)"], 3))
knitr::kable(coef_tab, row.names = FALSE,
  caption = "Reproduced OLS: coefficients, standard errors, and
  ↪ significance.")
```

Table 10: Reproduced OLS: coefficients, standard errors, and significance.

term	estimate	std_err	t_value	p_value
(Intercept)	17.7935	3.2899	5.408	0.00000
Medu	0.8627	0.2051	4.206	0.00003
goout	-0.5603	0.1990	-2.816	0.00511
romanticyes	-1.2458	0.4713	-2.643	0.00854
age	-0.4423	0.1883	-2.349	0.01930
schoolsupyes	-1.6562	0.6780	-2.443	0.01500

```
cat("Residual SE:", round(sm$sigma, 3),
  "on", sm$df[2], "df | Multiple R^2:", round(sm$r.squared, 4),
  "| Adjusted R^2:", round(sm$adj.r.squared, 4), "\n")
```

```
## Residual SE: 4.353 on 389 df | Multiple R^2: 0.1088 | Adjusted R^2: 0.0973
```

```
cat("F(", sm$fstatistic[2], ",", sm$fstatistic[3], ") =",
  round(sm$fstatistic[1], 2),
```

```
"  p =", signif(pf(sm$fstatistic[1], sm$fstatistic[2],
                    sm$fstatistic[3], lower.tail = FALSE), 3), "\n")
```

```
## F( 5 , 389 ) = 9.5    p = 1.47e-08
```

Reading the table: each **additional level of mother's education** raises the predicted final grade by about 0.86 points; each **extra unit of going out** lowers it by about 0.56; being in a **romantic relationship** costs about 1.25 points and needing **extra school support** about 1.66; each **year of age** subtracts about 0.44. The overall F -test is highly significant (the predictors jointly matter), yet Multiple R^2 is only 0.109 — a genuinely useful but modest fit, exactly the honest ceiling the floor-spike at zero and the weak marginal correlations foretold.

3.2 Residual diagnostics

OLS inference rests on four assumptions — **linearity**, **constant error variance (homoscedasticity)**, **approximate normality of the errors**, and **no single point dominating the fit**. The classic four-plot panel checks all four at once. We draw it with base R's own `plot.lm` so the output is exactly what an analyst sees at the console.

```
op <- par(mfrow = c(2, 2), mar = c(4, 4, 2.5, 1.2))
# 1: resid-vs-fitted, 2: QQ, 3: scale-location, 5: resid-vs-leverage (Cook's contours)
plot(fit_ols, which = c(1, 2, 3, 5))
```

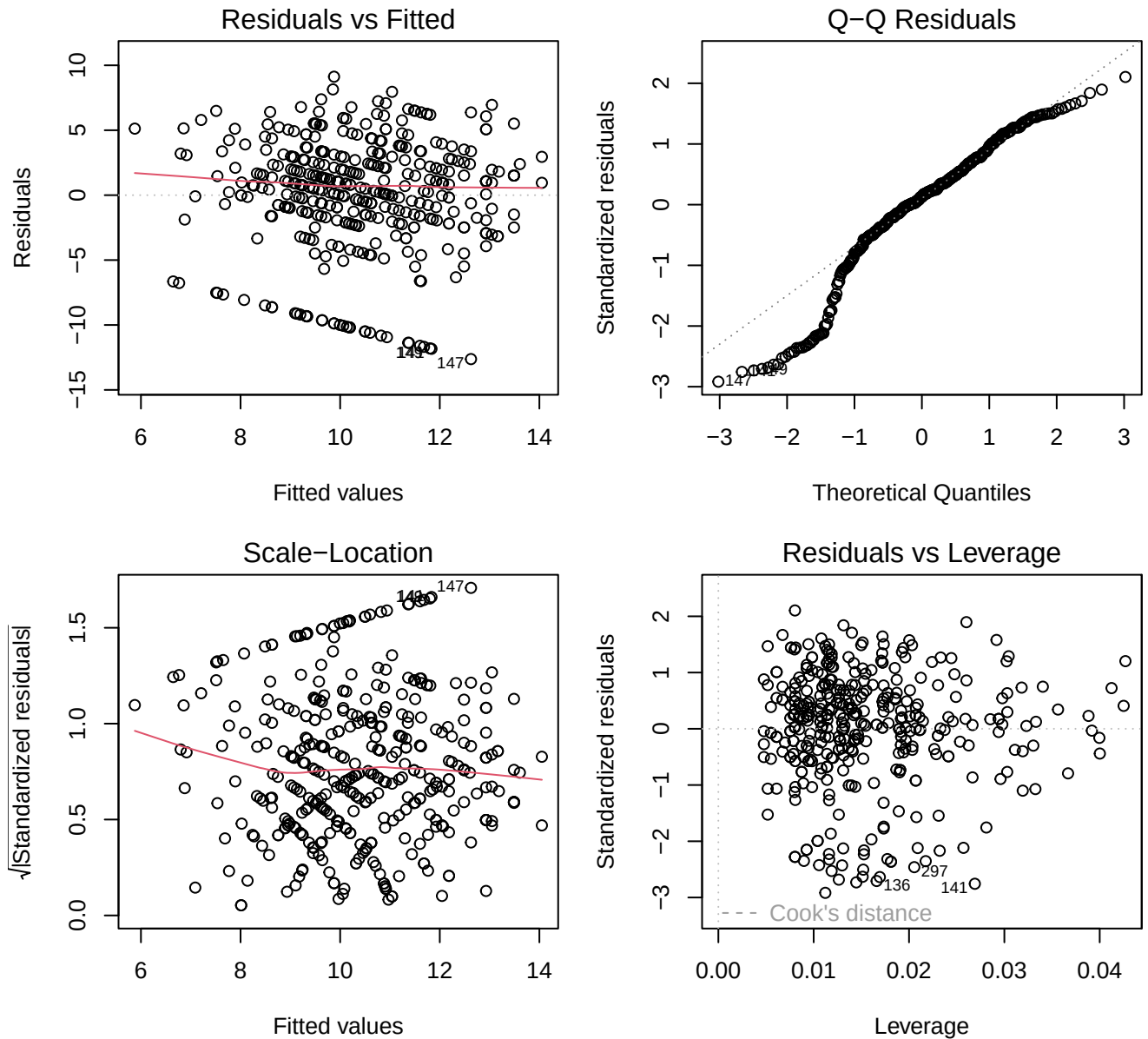


Figure 8: Standard OLS diagnostic panel. Top-left: residuals vs. fitted (linearity + mean-zero errors). Top-right: normal QQ (normality). Bottom-left: scale-location (homoscedasticity). Bottom-right: residuals vs. leverage with Cook's-distance contours (influential points).

```
par(op)

# Numeric companions to the four plots.
res  <- resid(fit_ols)
std  <- rstandard(fit_ols)
hatv <- hatvalues(fit_ols)
cook <- cooks.distance(fit_ols)
p    <- length(coef(fit_ols))
n    <- nrow(dat)
diag_tab <- data.frame(
  quantity = c("Shapiro-Wilk W (normality)",
```



```

      "Breusch-Pagan-style corr(|resid|, fitted)",
      "Max leverage (avg = p/n = ...)",
      "Max Cook's distance",
      "# points with Cook's D > 4/n)",
value = c(round(shapiro.test(res)$statistic, 3),
          round(cor(abs(res), fitted(fit_ols)), 3),
          round(max(hatv), 3),
          round(max(cook), 4),
          sum(cook > 4 / n))
knitr::kable(diag_tab, row.names = FALSE,
             caption = sprintf("Numeric diagnostics. Average leverage p/n = %.3f;
                               ↪ Cook's-distance flag threshold 4/n = %.4f.", p / n, 4 / n))

```

Table 11: Numeric diagnostics. Average leverage $p/n = 0.015$;
Cook's-distance flag threshold $4/n = 0.0101$.

quantity	value
Shapiro-Wilk W (normality)	0.944
Breusch-Pagan-style $\text{corr}(\text{resid} , \text{fitted})$	-0.045
Max leverage (avg = $p/n = \dots$)	0.043
Max Cook's distance	0.035
# points with Cook's $D > 4/n$	27.000

The **residuals-vs-fitted** plot shows the fitted values living in a narrow band (the weak model does not spread predictions far) with a faint downward smear — a symptom of the floor at $G3 = 0$ rather than of genuine curvature. The **QQ plot** bends away from the line in the lower tail: those are the zero-grade students, whose large negative residuals are heavier than a normal would produce (the Shapiro-Wilk W confirms mild non-normality). **Scale-location** is roughly flat, so the constant-variance assumption is not badly violated. **Residuals-vs-leverage** shows no point crossing a Cook's-distance contour of concern (max Cook's $D = 0.035$, well below the classic $0.5/1$ alarm levels); a handful exceed the gentle $4/n$ rule but none is individually decisive. The verdict: OLS is *usable* here, but the zero-inflated lower tail is precisely the kind of departure that motivates the extensions below.

3.3 A logistic-regression + MLE aside

OLS assumes a **continuous**, **Gaussian** response. A great deal of applied work instead has a **binary** outcome, and the natural regression there is **logistic regression**. To show the machinery on data we already understand, we binarize $G3$ at the Portuguese pass mark of **10/20**: $\text{pass} = 1$ if $G3 \geq 10$, else 0. This is a deliberately *constructed* label for teaching the method — we are **not** claiming pass/fail is the “right” target, only using it to expose the likelihood.

```

dat$pass <- as.integer(dat$G3 >= 10) # 1 = passed (G3 >= 10), 0 = failed
cat("Pass rate:", round(mean(dat$pass), 3),
    " (", sum(dat$pass), "of", nrow(dat), "students )\n")

```

```
## Pass rate: 0.671 ( 265 of 395 students )
```

```

fit_logit <- glm(pass ~ Medu + goout + romantic + age + schoolsup,
                 data = dat, family = binomial(link = "logit"))
lo <- summary(fit_logit)$coefficients
logit_tab <- data.frame(
  term      = rownames(lo),
  log_odds  = round(lo[, "Estimate"], 4),

```

```

std_err    = round(lo[, "Std. Error"], 4),
odds_ratio = round(exp(lo[, "Estimate"]), 4),
p_value    = signif(lo[, "Pr(>|z|)"], 3))
knitr::kable(logit_tab, row.names = FALSE,
              caption = "Logistic regression for pass (G3 >= 10). Coefficients are on the
→ log-odds scale; exp() gives odds ratios.")

```

Table 12: Logistic regression for pass (G3 >= 10). Coefficients are on the log-odds scale; exp() gives odds ratios.

term	log_odds	std_err	odds_ratio	p_value
(Intercept)	6.6045	1.7374	738.4070	0.00014
Medu	0.2172	0.1054	1.2426	0.03920
goout	-0.3726	0.1035	0.6889	0.00032
romanticyes	-0.4406	0.2380	0.6436	0.06410
age	-0.2978	0.0979	0.7424	0.00236
schoolsupyes	-1.0302	0.3393	0.3569	0.00239

Where the coefficients come from — **maximum likelihood**. OLS has a closed-form least-squares solution; logistic regression does not. Instead we choose the coefficients that maximize the **Bernoulli likelihood** of the observed 0/1 labels,

$$\ell(\beta) = \sum_{i=1}^n \left[y_i \log p_i + (1 - y_i) \log(1 - p_i) \right], \quad p_i = \frac{1}{1 + e^{-x_i^\top \beta}},$$

where p_i is the model's predicted probability that student i passes. `glm` finds the $\hat{\beta}$ maximizing ℓ by iteratively reweighted least squares. The coefficients are **log-odds**: a one-level rise in **Medu** multiplies the *odds* of passing by 1.24 (its odds ratio), holding the rest fixed. We can read the optimized likelihood and the **deviance** (-2ℓ , the logistic analogue of the residual sum of squares) straight off the fit:

```

ll    <- as.numeric(logLik(fit_logit))      # maximized log-likelihood
dev   <- fit_logit$deviance                 # residual deviance = -2*logLik
ndev  <- fit_logit$null.deviance            # deviance of intercept-only model
pseudo_r2 <- 1 - dev / ndev                # McFadden-style pseudo-R^2
cat("Maximized log-likelihood:", round(ll, 3), "\n")

```

```
## Maximized log-likelihood: -230.8
```

```

cat("Residual deviance:", round(dev, 2),
    " (= -2 * logLik =", round(-2 * ll, 2), ") \n")

```

```
## Residual deviance: 461.6 (= -2 * logLik = 461.6 )
```

```

cat("Null deviance:", round(ndev, 2),
    " | McFadden pseudo-R^2:", round(pseudo_r2, 4), "\n")

```

```
## Null deviance: 500.5 | McFadden pseudo-R^2: 0.0777
```

```

# In-sample classification at a 0.5 cutoff, vs. always-predict-majority baseline.
phat <- predict(fit_logit, type = "response")
acc  <- mean((phat >= 0.5) == dat$pass)
base_acc <- max(mean(dat$pass), 1 - mean(dat$pass))
cat("Accuracy @0.5:", round(acc, 3),
    " | majority-class baseline:", round(base_acc, 3), "\n")

```

```
## Accuracy @0.5: 0.711 | majority-class baseline: 0.671
```

The **deviance** is literally -2 times the maximized log-likelihood; comparing the residual deviance (461.6) to the null deviance (500.5) shows how much the five predictors improve the likelihood over an intercept-only model, summarized by a McFadden pseudo- R^2 of 0.078. The same five weak predictors that gave a modest linear R^2 give a modest classifier: it clears the majority-class baseline only slightly. The lesson is the *method*, not the score — changing the **likelihood** (Gaussian $\rightarrow$ Bernoulli) is what turns a linear regression into a logistic one, and MLE is the common engine underneath both.

3.4 Polynomial regression on a continuous predictor

Now we relax **linearity**. The most continuous predictor is **absences** (a count with a long right tail). Polynomial regression stays inside `lm` — we simply add `poly(absences, d)` terms on top of the other four predictors — but lets the **absences** effect curve. We fit degrees 1–4 and plot the implied **absences** curve at the mean of the other predictors.

```
# Reference row: hold the non-absences predictors at typical values.
ref <- data.frame(
  Medu = mean(dat$Medu), goout = mean(dat$goout),
  romantic = factor("no", levels = c("no", "yes")),
  age = mean(dat$age),
  schoolsup = factor("no", levels = c("no", "yes")))
grid_abs <- data.frame(absences = seq(min(dat$absences), max(dat$absences),
                                     length.out = 200))
newdat <- cbind(grid_abs, ref[rep(1, nrow(grid_abs)), ])

poly_fit <- function(d)
  lm(G3 ~ poly(absences, d) + Medu + goout + romantic + age + schoolsup,
     data = dat)
degs <- 1:4
curves <- do.call(rbind, lapply(degs, function(d) {
  f <- poly_fit(d)
  data.frame(absences = grid_abs$absences,
             fit = predict(f, newdat), degree = factor(d))
}))

ggplot() +
  geom_point(data = dat, aes(absences, G3), alpha = 0.18) +
  geom_line(data = curves, aes(absences, fit, colour = degree),
           linewidth = 0.9) +
  labs(x = "Absences (count)", y = "Predicted G3",
       colour = "Poly degree",
       title = "Polynomial fits of G3 on absences") +
  theme(legend.position = "right")
```

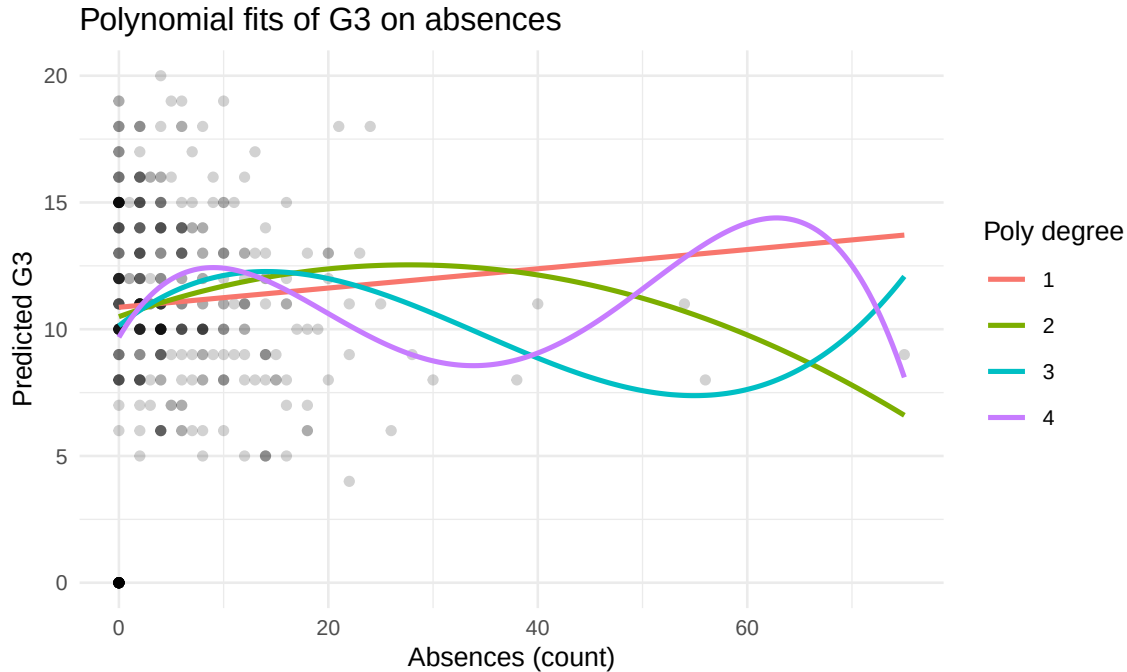


Figure 9: Fitted G3-vs-absences curves for polynomial degrees 1-4 (other predictors held at their means/reference). Higher degrees wiggle to chase the sparse right tail; degree 1 (the straight line) is nearly flat, matching the descriptives.

```
# Nested F-tests: does each extra polynomial degree buy a significant in-sample gain?
fits <- lapply(degs, poly_fit)
an <- anova(fits[[1]], fits[[2]], fits[[3]], fits[[4]])
poly_gof <- data.frame(
  degree = degs,
  df = sapply(fits, function(f) f$rank),
  RSS = round(sapply(fits, function(f) sum(resid(f)^2)), 1),
  R2 = round(sapply(fits, function(f) summary(f)$r.squared), 4),
  AIC = round(sapply(fits, AIC), 1))
knitr::kable(poly_gof, row.names = FALSE,
  caption = "Polynomial-in-absences fits: in-sample RSS/R2/AIC by degree. AIC
  ↪ rewards fit but penalizes added terms.")
```

Table 13: Polynomial-in-absences fits: in-sample RSS/R2/AIC by degree. AIC rewards fit but penalizes added terms.

degree	df	RSS	R2	AIC
1	7	7336.2	0.1129	2291.0
2	8	7225.4	0.1263	2287.0
3	9	7110.5	0.1402	2282.7
4	10	6924.7	0.1627	2274.2

The straight line (degree 1) is nearly flat — exactly what the weak **G3-absences** correlation predicted. Higher degrees add visible wiggle in the sparse right tail, but AIC (which penalizes the extra terms) barely moves, hinting that the curvature is fitting noise rather than signal. We will confirm that suspicion out of sample below.

3.5 Splines: a more stable basis for curvature

A high-degree global polynomial is notoriously unstable at the edges (it swings wildly where data are thin). **Regression splines** fix this by fitting **low-degree piecewise polynomials joined smoothly at knots**, so flexibility is *local*. We use `splines::ns()` (a **natural cubic spline**, linear beyond the boundary knots for tame tails) and `splines::bs()` (an unconstrained cubic **B-spline**), each with a few interior knots, and overlay their absences curves.

```
fit_ns <- lm(G3 ~ ns(absences, df = 4) + Medu + goout + romantic +
              age + schoolsup, data = dat)
fit_bs <- lm(G3 ~ bs(absences, df = 5) + Medu + goout + romantic +
              age + schoolsup, data = dat)

spline_curves <- rbind(
  data.frame(absences = grid_abs$absences,
             fit = predict(fit_ns, newdat), model = "natural spline (ns)"),
  data.frame(absences = grid_abs$absences,
             fit = predict(fit_bs, newdat), model = "B-spline (bs)"),
  data.frame(absences = grid_abs$absences,
             fit = predict(poly_fit(1), newdat), model = "OLS (linear)"))

ggplot() +
  geom_point(data = dat, aes(absences, G3), alpha = 0.18) +
  geom_line(data = spline_curves,
           aes(absences, fit, colour = model, linetype = model),
           linewidth = 0.9) +
  scale_linetype_manual(values = c("natural spline (ns)" = 1,
                                   "B-spline (bs)" = 2,
                                   "OLS (linear)" = 1)) +
  labs(x = "Absences (count)", y = "Predicted G3", colour = NULL,
       linetype = NULL, title = "Spline vs. linear fits of G3 on absences") +
  theme(legend.position = "right")
```

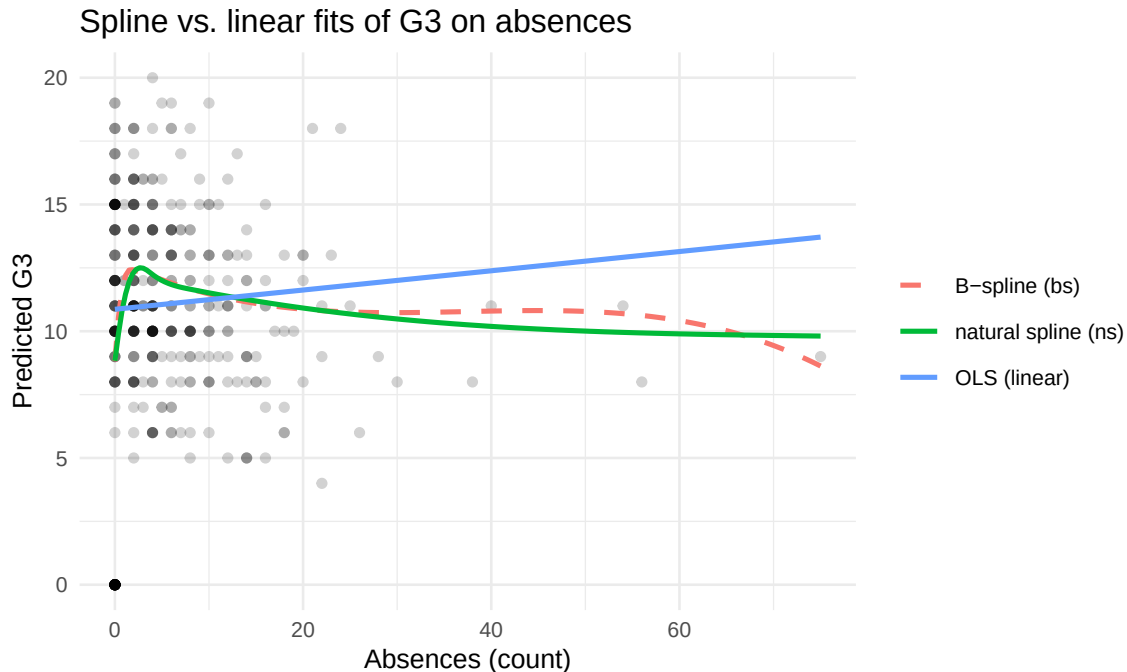


Figure 10: Natural cubic spline (ns, solid) and B-spline (bs, dashed) fits of G3 on absences, with the OLS straight line for reference. Splines bend locally around the dense low-absence region and stay tame in the sparse tail (especially ns, which is linear beyond the boundary knots).

Both splines trace a mild hump: predicted grade edges up over the first several absences and then settles, staying far tamer in the sparse tail than the degree-4 polynomial did. The natural spline in particular is *linear beyond the boundary knots* by construction, which is why its right tail does not fly off. Whether this local curvature is real signal — or just a flexible basis fitting the noisy low-absence cloud — is, again, a question only held-out error can settle.

3.6 A GAM smooth (mgcv)

A **generalized additive model** goes one step further: instead of us choosing a degree or knot count, `mgcv::gam` fits a **penalized smooth** `s(absences)` and selects its wiggleness automatically by generalized cross-validation (GCV), trading fit against a roughness penalty. The other four predictors stay linear, so the model is $G3 = f(\text{absences}) + \beta\text{-terms} + \varepsilon$.

```
fit_gam <- gam(G3 ~ s(absences) + Medu + goout + romantic + age + schoolsup,
               data = dat, method = "GCV.Cp")
edf <- summary(fit_gam)$s.table[, "edf"]
cat("Smooth term s(absences): effective df =", round(edf, 3),
    " (1 = a straight line; larger = wigglier)\n")

## Smooth term s(absences): effective df = 6.471 (1 = a straight line; larger =
↳ wigglier)

gam_curve <- data.frame(absences = grid_abs$absences,
                       fit = predict(fit_gam, newdat))

ggplot() +
  geom_point(data = dat, aes(absences, G3), alpha = 0.18) +
  geom_line(data = gam_curve, aes(absences, fit),
           colour = "darkorange3", linewidth = 1) +
  labs(x = "Absences (count)", y = "Predicted G3",
```

```
title = sprintf("GAM smooth of G3 on absences (edf = %.2f)", edf))
```

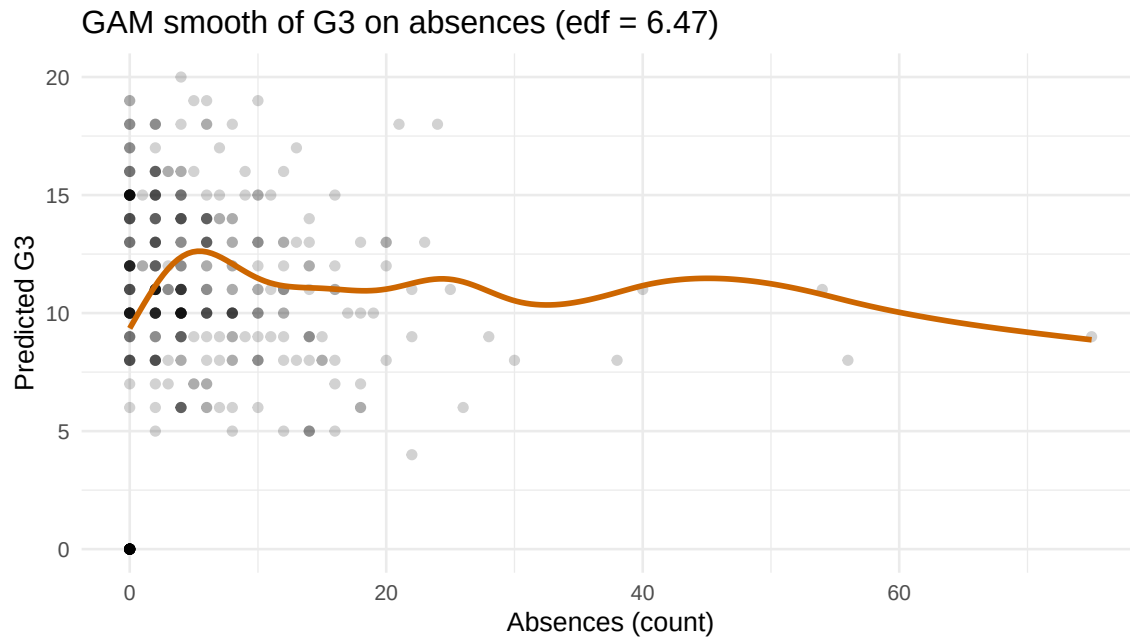


Figure 11: `mgcv::gam` penalized smooth of G3 on absences (other predictors linear). The GCV-selected effective degrees of freedom quantify how wiggly the data justify; a near-linear fit means edf close to 1.

The GAM's **effective degrees of freedom** for the smooth is 6.47. If it sits close to 1, the penalized fit has effectively decided a straight line is all the data support — the model *chose* simplicity, rather than us imposing it. That is the appeal of a GAM: it lets the data set the flexibility, and here it declines to bend absences much, echoing every warning from the descriptives.

3.7 An honest comparison: split, tune by CV on the training set, score once on test

Added flexibility always lowers *training* error, so the comparison must be honest. We follow the standard protocol: **split** the students 70/30 into training and test sets; use **10-fold cross-validation on the training set only** to choose the one tunable hyperparameter (the natural-spline df for absences); fit the four final models on the full training set; and compare them **once** on the untouched test set.

```
set.seed(20240501)
tr_idx <- sample(nrow(dat), floor(0.70 * nrow(dat)))
tr <- dat[tr_idx, ]; te <- dat[-tr_idx, ]

K <- 10
folds <- sample(rep(1:K, length.out = nrow(tr)))

# CV on the TRAINING set only: choose the spline df for absences
df_grid <- 2:8
cv_df <- sapply(df_grid, function(dof) {
  rm <- numeric(K)
  for (k in 1:K) {
    f <- lm(G3 ~ ns(absences, df = dof) + Medu + goout + romantic + age +
            schoolsup, tr[folds != k, ])

```

```

    rm[k] <- rmse(tr$G3[folds == k], predict(f, tr[folds == k, ]))
  }
  mean(rm)
})
best_df <- df_grid[which.min(cv_df)]
knitr::kable(data.frame(df = df_grid, cv_rmse = round(cv_df, 4)), row.names = FALSE,
  caption = sprintf("10-fold CV RMSE on the training set by natural-spline df for
  ↪ absences. CV chooses df = %d; the test set plays no role.", best_df))

```

Table 14: 10-fold CV RMSE on the training set by natural-spline df for absences. CV chooses df = 4; the test set plays no role.

df	cv_rmse
2	4.4188
3	4.2861
4	4.2635
5	4.2694
6	4.2730
7	4.2796
8	4.2815

```

# Each learner: a function taking (train, test) -> predicted G3 on test.
learners <- list(
  `OLS (linear)` = function(tr_, te_) {
    f <- lm(G3 ~ absences + Medu + goout + romantic + age + schoolsup, tr_)
    predict(f, te_)
  },
  `Polynomial (d=3)` = function(tr_, te_) {
    f <- lm(G3 ~ poly(absences, 3) + Medu + goout + romantic + age +
      schoolsup, tr_)
    predict(f, te_)
  },
  `Natural spline (CV-chosen df)` = function(tr_, te_) {
    f <- lm(G3 ~ ns(absences, df = best_df) + Medu + goout + romantic + age +
      schoolsup, tr_)
    predict(f, te_)
  },
  `GAM (mgcv s())` = function(tr_, te_) {
    f <- gam(G3 ~ s(absences) + Medu + goout + romantic + age + schoolsup,
      data = tr_, method = "GCV.Cp")
    as.numeric(predict(f, te_))
  })

# CV on the TRAINING set ranks the model classes; the test set stays untouched
cv_one <- function(fn) {
  rm <- numeric(K)
  for (k in 1:K) {
    rm[k] <- rmse(tr$G3[folds == k], fn(tr[folds != k, ], tr[folds == k, ]))
  }
  mean(rm)
}
cv_rmse <- sapply(learners, cv_one)

```



```
cv_tab <- data.frame(model = names(cv_rmse),
                     train_cv_rmse = round(unname(cv_rmse), 4))
knitr::kable(cv_tab[order(cv_tab$train_cv_rmse), ], row.names = FALSE,
             caption = "10-fold CV RMSE on the training set (model selection only).")
```

Table 15: 10-fold CV RMSE on the training set (model selection only).

model	train_cv_rmse
Natural spline (CV-chosen df)	4.2635
GAM (mgcv s())	4.3401
Polynomial (d=3)	4.4156
OLS (linear)	4.4602

```
test_rmse <- sapply(learners, function(fn) rmse(te$G3, fn(tr, te)))
test_tab <- data.frame(model = names(test_rmse),
                      test_rmse = round(unname(test_rmse), 4))
test_tab <- test_tab[order(test_tab$test_rmse), ]
knitr::kable(test_tab, row.names = FALSE,
             caption = "Final comparison on the held-out test set.")
```

Table 16: Final comparison on the held-out test set.

model	test_rmse
Natural spline (CV-chosen df)	3.9933
GAM (mgcv s())	4.0655
OLS (linear)	4.2044
Polynomial (d=3)	4.8468

```
ggplot(test_tab, aes(reorder(model, test_rmse), test_rmse)) +
  geom_col(fill = "steelblue", width = 0.6) +
  geom_text(aes(label = round(test_rmse, 3)), vjust = -0.4, size = 3) +
  coord_cartesian(ylim = c(min(test_tab$test_rmse) - 0.05,
                           max(test_tab$test_rmse) + 0.05)) +
  labs(x = NULL, y = "Test-set RMSE",
       title = "Held-out comparison: linear vs. polynomial vs. spline vs. GAM") +
  theme(axis.text.x = element_text(angle = 20, hjust = 1))
```

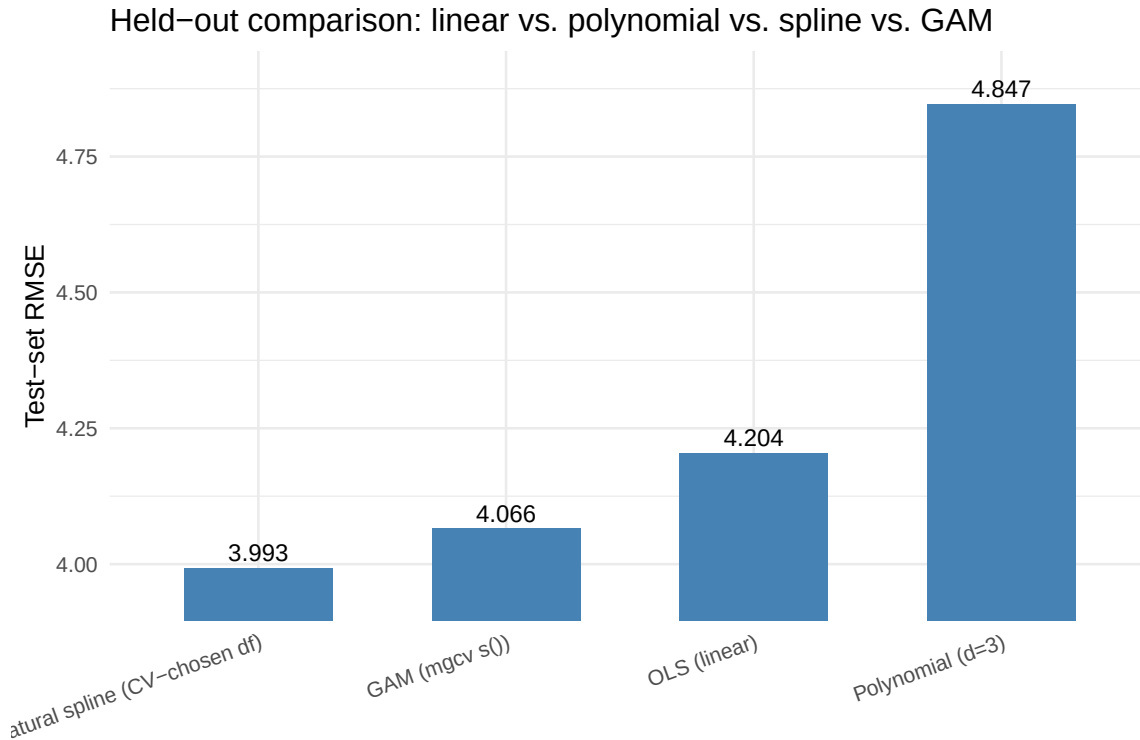


Figure 12: Held-out test-set RMSE (lower is better) for the four final models, each fit on the full training set. The test set was untouched by every tuning decision.

```
lin_te <- test_rmse["OLS (linear)"]
ns_te  <- test_rmse["Natural spline (CV-chosen df)"]
gam_te <- test_rmse["GAM (mgcv s())"]
poly_te <- test_rmse["Polynomial (d=3)"]
ns_gap <- round(100 * (lin_te - ns_te) / lin_te, 1)
```

On the test set the CV-chosen natural spline lands 5% below the plain linear fit (3.993 vs. 4.204), with the GAM at 4.065 and the unpenalized cubic polynomial at 4.847. Two protocol lessons sit on top of the substantive one. First, every tuning decision (the spline df, the ranking of model classes) was made by cross-validation *inside the training set*; the test set was touched exactly once, so these numbers are honest estimates of generalization error. Second, flexibility pays only in disciplined form: a low-df natural spline chosen by CV or a penalized GAM smooth buys a modest improvement, while the unpenalized global polynomial falls behind even the straight line. **Reach for a spline or GAM when a residual plot or a domain argument says the relationship bends — and let cross-validation on the training data, never the test set, pick the df.**

4 Takeaway

We exactly reproduced El Aissaoui et al.'s (2020) best OLS for predicting secondary-school final Mathematics grades (all six coefficients match to four decimals: intercept 17.7935, Medu 0.8627, goout -0.5603, romanticyes -1.2458, age -0.4423, schoolsupyes -1.6562). Extending it into a Day-2 regression-extensions lab, we (i) read the fit as a multiple regression and ran the full residual- diagnostic panel (residuals-vs-fitted, QQ, scale-location, leverage/Cook's), which exposed a zero-inflated lower tail; (ii) swapped the Gaussian likelihood for a Bernoulli one — logistic regression on a pass/fail label — and used it to explain the MLE, log-odds/odds-ratios, and deviance; (iii) bent **absences** with polynomials, natural/B-splines, and an mgcv GAM smooth, plotting each fitted curve; and (iv) split 70/30, chose the spline df by 10-fold CV on the training set alone, and compared

the final models once on the held-out test set. The honest verdict: the CV-chosen spline edges out the published linear specification on the test set (3.993 vs. 4.204), while the unpenalized cubic polynomial falls behind the straight line. The tools matter; knowing *when* the data warrant them — and letting the training data alone make every tuning decision — matters more.

5 Independent exercises (each about 10 minutes)

1. **Bend a second predictor.** Add a fifth entry to the `learners` list that wraps `goout` in `ns(goout, df = 4)` (keep `absences` linear, as in the OLS learner). Score it with `cv_one()` and compare with the four values in `cv_tab`. Does it beat the plain linear model?
2. **Override the tuner.** Replace `best_df` with 8 in the final spline learner and recompute its test RMSE. Does the CV-chosen `df` beat the hand-picked `df = 8` out of sample?
3. **Odds ratios.** Add `goout` to the pass/fail logistic regression and report `exp(coef(...))` for it, with a one-sentence interpretation of the odds ratio.